# Socket options

- In addition to binding a socket to a local address or connecting it to a destination address, application programs need a method to control the socket.
- For example, when using protocols that use time out and retransmission, the application program may want to obtain or set the time-out parameters.
- An application program may also want to control the allocation of buffer space, determine if the socket allows transmission of broadcast, or control processing of out-of-band data.

# SO_KEEPALIVE

- The SO_KEEPALIVE socket option is valid only for protocols that support the notion of keep-alive (connection-oriented protocols).
- In Java, the SO_KEEPALIVE socket option is used to enable or disable the TCP keep-alive mechanism for a socket. The TCP keep-alive mechanism is designed to detect if a connection is still alive and functioning. It does this by periodically sending small keep-alive packets to the remote end of the connection and waiting for a response. If no response is received after a certain period of time, the connection is considered dead, and an error is reported.
- Enabling SO_KEEPALIVE can be useful in scenarios where the network connection between two parties might become idle for an extended period, and you want to detect if the connection is still active without having to send actual data.

# SO_LINGER

- This option specifies how the close function operates for a connection-oriented protocol (for TCP, but not for UDP). By default, close returns immediately, but if there is any data still remaining in the socket send buffer, the system will try to deliver the data to the peer.
- The SO_LINGER option allows you to specify how the socket should behave when it is closed with pending data. It takes an array of two integers as its argument, representing the onoff and linger values:
- onoff: An integer value (0 or 1) that enables or disables the SO_LINGER option. If onoff is set to 1, it means the SO_LINGER option is enabled. If set to 0, the SO_LINGER option is disabled.

# SO_SNDBUF

- The SO_SNDBUF socket option allows you to adjust the size of the send buffer to optimize the performance of your application based on the network characteristics and the data transmission requirements.

- To set the size of the send buffer, you use the setSendBufferSize(int size) method on the Socket or ServerSocket objects. To retrieve the current size of the send buffer, you can use the getSendBufferSize() method.

# SO_RCVBUF

- The SO_RCVBUF socket option allows you to adjust the size of the receive buffer to optimize the performance of your application based on the network characteristics and the data reception requirements.

- To set the size of the receive buffer, you use the setReceiveBufferSize(int size) method on the Socket or ServerSocket objects. To retrieve the current size of the receive buffer, you can use the getReceiveBufferSize() method.

# TCP_NODELAY

- In socket programming, the TCP_NODELAY socket option is used to control the Nagle's algorithm for TCP connections. Nagle's algorithm is designed to improve network efficiency by reducing the number of small packets sent over the network. It does this by delaying the transmission of small packets and combining them into larger packets before sending them out.
- When the TCP_NODELAY option is enabled (set to true), Nagle's algorithm is turned off, and the socket will try to send data as soon as possible without waiting for more data to accumulate. This can help reduce the latency and improve the responsiveness of the communication.